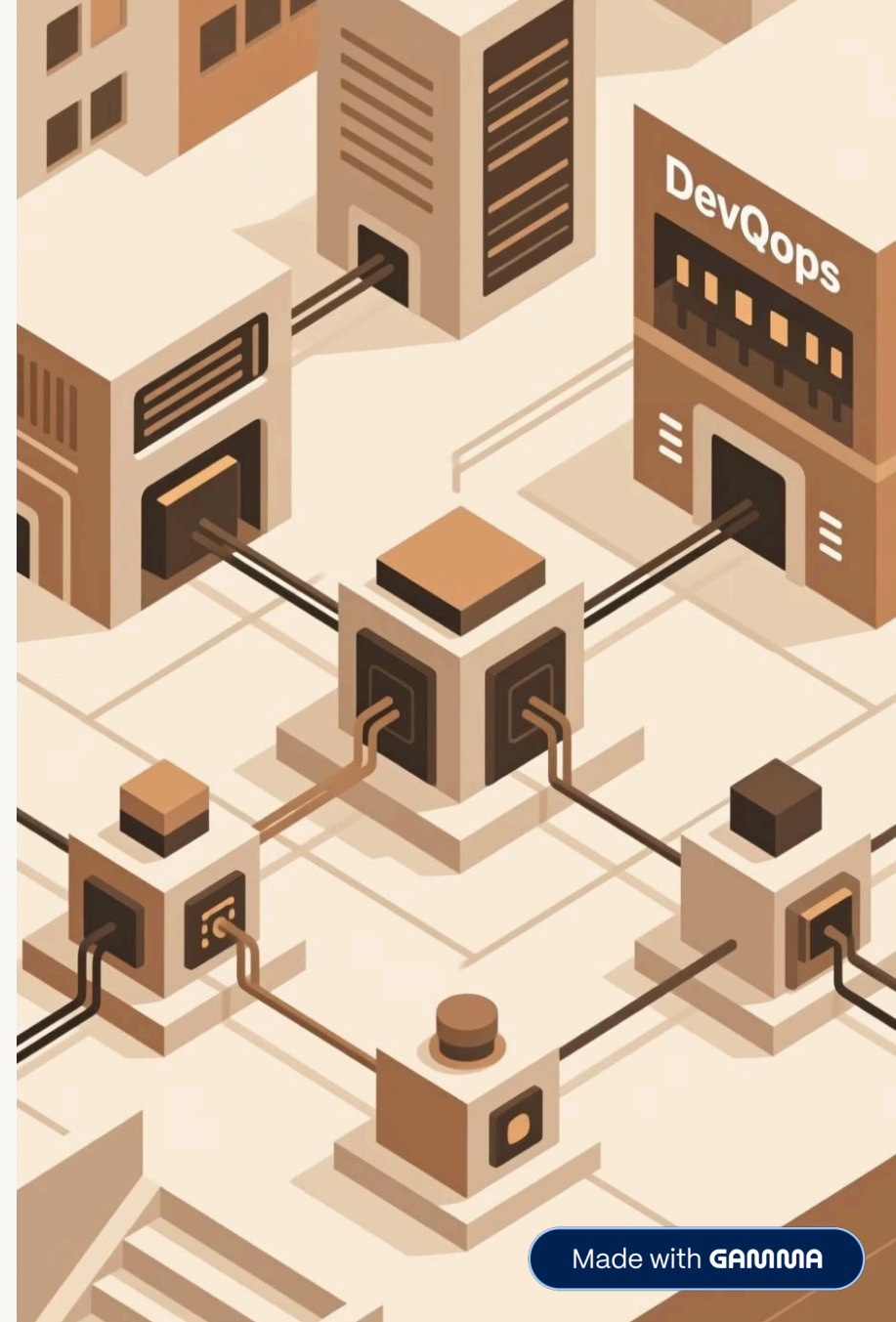


Ansible Roles & Collections

Mastering modular, reusable automation for scalable infrastructure management



What Are Ansible Roles?

Roles provide a standardized structure to organize your Ansible automation components—tasks, variables, templates, handlers, and modules—into clean, maintainable units.

Key benefits include:

- Reusability across multiple projects and teams
- Parameterization through variables for flexible customization
- Easy packaging and sharing within organizations
- Separation of concerns for complex infrastructure

Common use cases: installing Java (JVM), PostgreSQL, web servers, or any repeatable configuration task.



Ansible Galaxy: Your Role Repository



Community Hub

A centralized platform for sharing and discovering Ansible roles and collections—similar to PyPI or Maven Central for dependency management.



CLI Management

Managed via the ansible-galaxy command-line tool for installing, creating, and publishing roles.



Version Control

Provides documentation, version management, and dependency tracking for all published roles.

```
ansible-galaxy role install geerlingguy.java
```

Using Roles in Playbooks

Roles are declared under the `roles:` section of a playbook. Variables can be passed to customize behavior, and the playbook automatically executes all internal role tasks.

Playbook Structure

```
---  
- hosts: webservers  
  roles:  
    - role: geerlingguy.java  
    java_packages:  
      - java-11-openjdk
```

Benefits

- Abstract implementation details
- Enable code reuse effortlessly
- Maintain consistency across environments
- Simplify complex automation workflows

Understanding Execution Order

01

Role Dependencies Execute First

Any dependencies defined in `meta/main.yml` run before the role itself, respecting tag and condition filters.

02

Roles Run in Listed Order

When both roles and tasks exist in a playbook, Ansible executes all roles first, in the sequence they're declared.

03

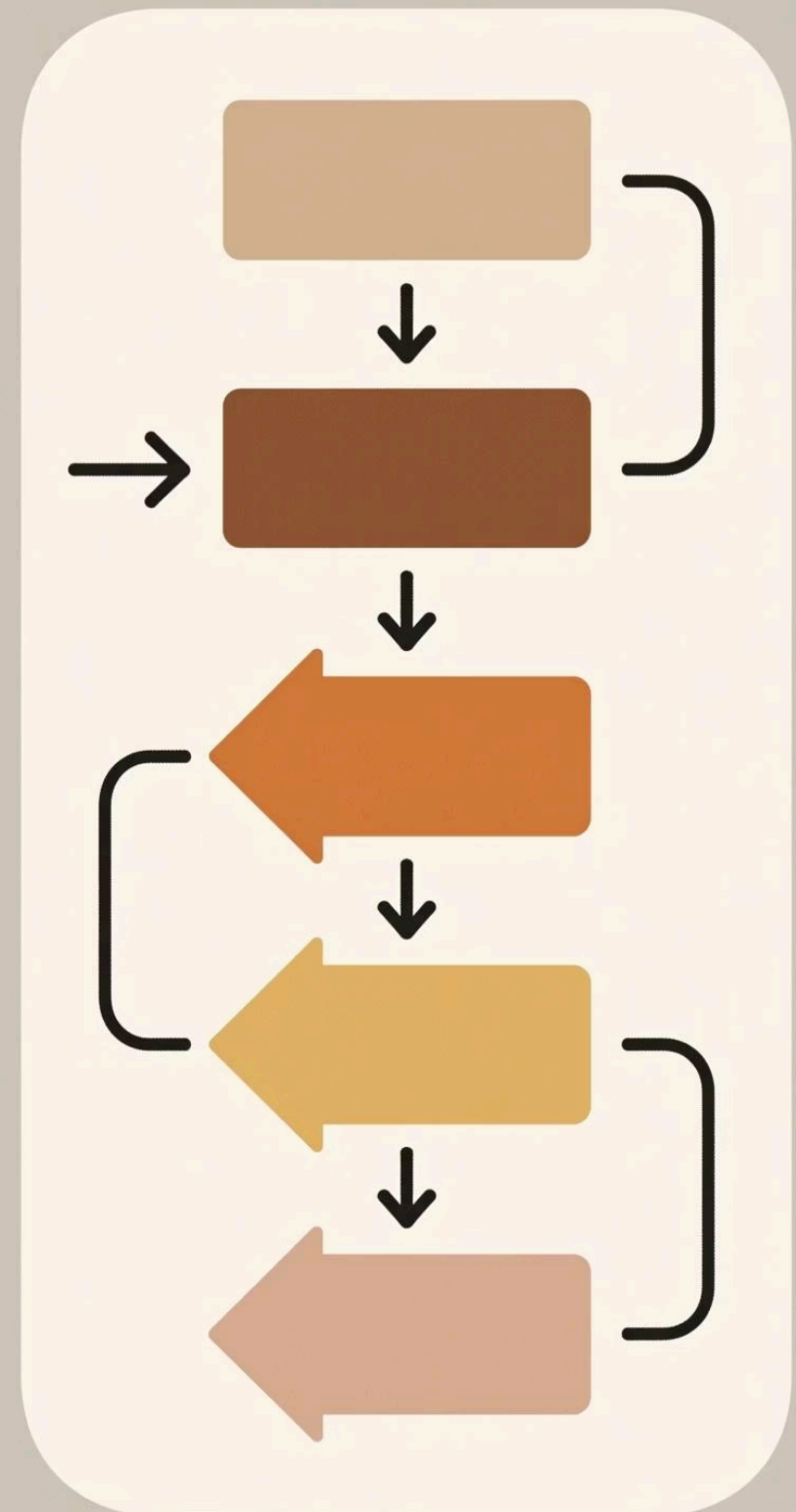
Play-Level Tasks Follow

After roles complete, play-level tasks execute sequentially in the order defined.

04

Dynamic Placement Available

Use `include_role` or `import_role` to place roles between tasks for customized execution flow.



Developing Custom Roles

When existing Galaxy roles don't meet your specific requirements or organizational policies, create your own. Let's build a simple role called `printfile` that displays remote file content.

1

Generate Scaffold

```
ansible-galaxy role init printfile
```

2

Standard Structure

Includes: tasks, vars, defaults, templates, files, handlers, meta, and tests directories.

3

Define Variables

Set defaults in `vars/main.yml`:
`show_file: /etc/hosts`



Handlers: Event-Driven Tasks

What Are Handlers?

Handlers are special tasks triggered only when notified by other tasks. They promote reusability and cleaner separation between actions and responses.

Key characteristics:

- Run once even if notified multiple times
- Triggered by multiple tasks if needed
- Defined in `handlers/main.yml`
- Ideal for service restarts and configuration reloads

Example Handler

```
---  
- name: Display file content  
  debug:  
    msg: "File changed!"  
  listen: "file changed"
```

Notify from Task

```
- name: Copy config  
  copy:  
    src: app.conf  
    dest: /etc/app.conf  
    notify: "file changed"
```

Sharing Roles Across Teams



Git Repositories

Recommended for enterprise version control. Provides security, versioning history, and persistent storage with full audit trails.



Ansible Galaxy / Automation Hub

Centralized sharing and discovery platform. Ideal for organization-wide or community-wide distribution with built-in documentation.



HTTP Servers

Simple file-based hosting for internal use. Straightforward setup but lacks version control and sophisticated management features.

Define sources in `requirements.yml` with paths to local files, Git repos, or remote URLs.



Ansible Collections: Beyond Roles

Collections bundle multiple types of Ansible content—roles, modules, plug-ins, and playbooks—into one distributable unit, providing a more scalable and modular approach than individual roles.

1 — Create Collection

```
ansible-galaxy collection init mynamespace.mycollection
```

2 — Build Package

```
ansible-galaxy collection build
```

Creates: `mynamespace-mycollection-1.0.0.tar.gz`

3 — Install Locally

```
ansible-galaxy collection install mynamespace-  
mycollection-1.0.0.tar.gz
```

4 — Use with FQCN

Reference via Fully Qualified Collection Name in playbooks

Key Takeaways

Roles = Single Responsibility

Each role should handle one task (Java install, Nginx config) for maintainability and clarity.

Collections = Multi-Component Packaging

When automation requires grouped functionalities (install + configure + secure), collections provide better structure than multiple roles.

Reusability Drives Efficiency

Both roles and collections enable sharing across projects, reducing duplication and standardizing infrastructure automation.

Master these patterns to build scalable, maintainable automation that grows with your infrastructure needs.

